

Sistem Operasi (INF-201)

Pertemuan 2: Arsitektur Kernel & Struktur Sistem Operasi

Tim Dosen Sistem Operasi

Program Studi Informatika – Universitas Bengkulu

Semester Ganjil 2026/2027

Tanggal Kompilasi: 8 September 2026



Capaian Pembelajaran (Sub-CPMK 2)

- Mahasiswa mampu menganalisis struktur internal dan arsitektur Kernel Sistem Operasi.
- Mahasiswa mampu membandingkan trade-off performa, keamanan, dan modularitas antara **Monolithic Kernel**, **Microkernel**, dan **Hybrid Kernel**.
- Mahasiswa memahami mekanisme *Loadable Kernel Modules (LKM)* untuk ekstensibilitas driver.
- Mahasiswa mampu menguraikan tahapan proses booting komputer modern (UEFI/BIOS → GRUB → Kernel Init → Systemd/PID 1).

Apa itu Kernel?

Definisi Kernel (OSTEP Ch. 2 & 4)

Komponen inti sistem operasi yang selalu dimuat di memori RAM fisik dan berjalan dengan hak akses tertinggi perangkat keras (*Ring 0 / Supervisor Mode*).

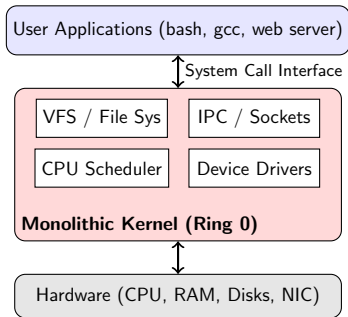
Tanggung Jawab Inti Kernel:

- **Manajemen Proses:** Scheduling, Context Switch, IPC.
- **Manajemen Memori:** Paging, Alokasi Alamat Virtual.
- **Manajemen Berkas:** VFS, i-node, buffer cache.
- **Device Drivers:** Abstraksi hardware I/O.

Hierarki Proteksi Hardware:

- **Ring 3 (User Mode):** Aplikasi web, compiler, GUI shell.
- **Ring 0 (Kernel Mode):** Instruksi CPU istimewa (CR3, interrupt handler, disable interrupts).

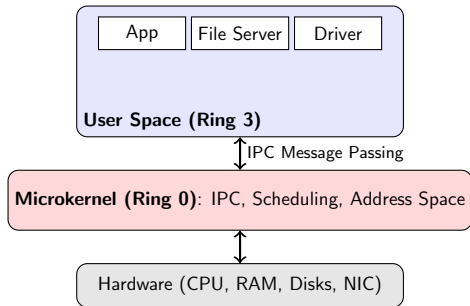
Arsitektur 1: Monolithic Kernel (Linux, FreeBSD)



Karakteristik Monolithic:

- Semua layanan OS berjalan dalam **satu address space** di Ring 0.
- **Keunggulan:** Sangat cepat (komunikasi antar fungsi via pointer call biasa tanpa overhead pesan IPC).
- **Kelemahan:** Kerentanan keamanan tinggi; bug pada satu driver dapat menyebabkan *Kernel Panic* / *Blue Screen*.

Arsitektur 2: Microkernel (seL4, QNX, Minix)



Karakteristik Microkernel:

- Hanya fungsionalitas minimalis yang berada di Ring 0.
- File system dan driver berjalan sebagai proses mandiri di Ring 3.
- **Keunggulan:** *Fault Tolerance* ekstrem (driver crash bisa di-restart otomatis tanpa reboot OS).
- **Kelemahan:** Performa lebih lambat akibat overhead context switch antar pesan IPC.

Arsitektur 3: Hybrid Kernel (macOS Darwin / Windows NT)

Konsep Hybrid (Kombinasi Monolithic & Microkernel)

Menggunakan arsitektur berbasis pesan dan abstraksi port Mach Microkernel, namun mengeksekusi subsistem File System (BSD/NTFS), Networking, dan Grafis di dalam satu ruang alamat kernel (Ring 0) demi performa maksimal.

Struktur XNU Kernel (macOS/iOS):

- **Mach Layer:** Abstraksi thread, tasks, IPC ports, virtual memory.
- **BSD Layer:** Antarmuka POSIX, permissions, VFS, BSD sockets.
- **I/O Kit:** Driver berorientasi objek (C++ subset) yang aman.

Struktur Windows NT:

- **Executive Layer:** Memory manager, Security reference monitor, I/O manager.
- **HAL (Hardware Abstraction Layer):** Memisahkan detail motherboard dari kernel utama.

Tabel Perbandingan Tipe Arsitektur Kernel

Dimensi Evaluasi	Monolithic (Linux)	Microkernel (seL4/QNX)	Hybrid (macOS/WinNT)
Performa I/O	Tertinggi (tanpa overhead context switch)	Menengah (overhead passing pesan IPC)	Tinggi (subsistem kritis tetap di Ring 0)
Keamanan / Isolasi	Rendah (1 bug driver = crash total)	Tertinggi (driver terisolasi di user space)	Menengah
Ukuran Kode Ring 0	Sangat Besar (> 30 Juta Baris)	Sangat Kecil (< 10.000 Baris)	Besar
Modularitas	Via LKM dinamis	Secara native berorientasi modular	Modular via Driver Framework
Domain Penggunaan	Server, Cloud, Android	Medis, Otomotif, Kedirgantaraan	Desktop PC, Laptop, Smartphone

Loadable Kernel Modules (LKM)

Bagaimana Linux Mengatasi Kekurangan Monolithic?

Linux menggunakan mekanisme **Loadable Kernel Modules (LKM)** sehingga fungsionalitas kernel dan driver dapat ditambahkan atau dihapus saat sistem sedang berjalan (*runtime*) tanpa perlu me-recompile kernel atau me-restart komputer.

Perintah Manajemen LKM Linux:

- `lsmod`: Menampilkan daftar modul aktif.
- `modinfo <modul>`: Melihat metadata modul.
- `insmod / modprobe`: Memuat modul ke RAM.
- `rmmmod`: Membongkar modul dari RAM.

Struktur Kode Modul C Sederhana:

- `int init_module(void)`: Dipanggil saat modul di-load.
- `void cleanup_module(void)`: Dipanggil saat modul di-unload.

Alur Booting Sistem Komputer Modern (UEFI → PID 1)



- ❶ **UEFI/Firmware:** Inisialisasi perangkat keras dasar (POST), membaca partisi EFI System Partition (ESP).
- ❷ **Bootloader (GRUB):** Menampilkan menu OS, memuat file citra kernel terkompresi (`vmlinuz`) dan root disk sementara (`initramfs`) ke memori RAM fisik.
- ❸ **Inisialisasi Kernel:** Kernel mengaktifkan proteksi CPU (Paging & MMU), memuat driver disk utama, dan me-mount partisi Root (/).
- ❹ **Spawning PID 1:** Kernel bertransisi ke User Space untuk menjalankan proses nomor 1 (`systemd`), yang kemudian meluncurkan seluruh service background dan login screen.

Studi Kasus Desain: Kapan Memilih Microkernel vs Monolithic?

Kasus 1: Alat Medis & Otomotif

Pilihan: Microkernel (seL4 / QNX / VxWorks)

- Sistem rem ABS mobil atau pompa insulin tidak boleh mati sesaat pun.
- Crash pada driver Bluetooth tidak boleh merusak sistem kendali rem mesin.
- Kode kernel yang kecil memungkinkan verifikasi matematika formal (*Formal Verification*).

Kasus 2: Cloud Server & Superkomputer

Pilihan: Monolithic (Linux)

- Menangani jutaan network packet per detik menuntut zero-overhead.
- Ekosistem hardware dan driver sangat kaya dan didukung komunitas luas.
- Skalabilitas multicore dan efisiensi throughput adalah prioritas utama (#1).

Tugas Membaca Mingguan (Reading Assignment)

Persiapan Pertemuan 3: Manajemen Proses & POSIX C API

Sebelum mengikuti kuliah minggu depan, mahasiswa wajib membaca:

- **[OSTEP]** Chapter 5: *Interlude: Process API* (hal. 39–48)
- **[Think OS]** Chapter 2: *Processes* (hal. 9–16)
- **[TLCL]** Chapter 10: *Processes* (*ps*, *top*, *kill*) (hal. 104–118)
- **[xv6 RISC-V]** Chapter 1: *Code: System Calls* (*fork*, *exec*, *exit*) (hal. 9–14)

Pertanyaan Pemicu Diskusi Mandiri

Mengapa fungsi `fork()` mengembalikan nilai 0 pada proses anak, sedangkan pada proses induk mengembalikan PID dari anak tersebut?